Microprocessor and Assembly Language

CSE 3321

Microprocessor Fundamentals

Presented By

Saifur Rahman

Lecturer

Dept. of CSE

NBIU

References

Assembly Language Programming and Organization of the IBM PC

By Ytha Yu and Charles Maru

Microcomputer Systems

Consists of:

- System Unit
- Keyboard
- Display Screen
- Disk Drivers

System Unit → The computer

Keyboard
Display Screen
Disk Drivers

→ I/O Devices
Peripherals

Microcomputer Systems (cont'd)

The computer circuit consists of three parts:

- CPU
- Memory
- I/O

In a microcomputer, the CPU is a single chip processor called **microprocessor**.

Fig 1: Microcomputer

Microcomputer Systems (cont'd)

- **Integrated-circuit (IC)** chips are used in the construction of computer circuits.
- IC circuits are known as digital circuits.
- Computers process information using discrete voltage levels, typically representing high voltage as '1' and low voltage as '0'.
- These '1's and '0's are known as **binary digits** or **bits**.
- All data processed by computers is represented as strings of these bits, forming what is called a **bit string**.

Motherboard

- The motherboard is a printed circuit board that is the foundation of a computer, located at the bottom of the computer case.
- It allocates power to the CPU, RAM, and all other computer hardware components.
- Most importantly, the motherboard allows hardware components to communicate with one another.

Fig 2: Motherboard

Bytes and Words

- Information processed by the computer is stored in its memory.
- The memory circuits are usually organized into groups that can store eight bits of data.
- A string of eight bits is called a **byte**.
- Each memory byte is identified by a number that is called **address**.
- The data stored in a memory byte are called its **contents**.
- Two bytes form a **word**.
- The address of a memory byte is fixed and differs from the address of any other memory byte in the computer.
- The contents of a memory byte are not unique and may change, as they represent the data currently stored.

Address	Contents
.	
.	
.	
.	. . . .
7	0 0 1 0 1 1 0 1
6	1 1 0 0 1 1 1 0
5	0 0 0 0 1 1 0 1
4	1 1 1 0 1 1 0 1
3	0 0 0 0 0 0 0 0
2	1 1 1 1 1 1 1 1
1	0 1 0 1 1 1 1 0
0	0 1 1 0 0 0 0 1

Fig 3: Memory Represented as Bytes

Bit Position

- Positions are numbered from **right to left**, starting with 0.
- In a word, bits 0 to 7 compose the **low byte**, while bits 8 to 15 make up the **high byte**.
- The low byte of a word originates from the memory byte with the lower address, and the high byte comes from the memory byte with the higher address.

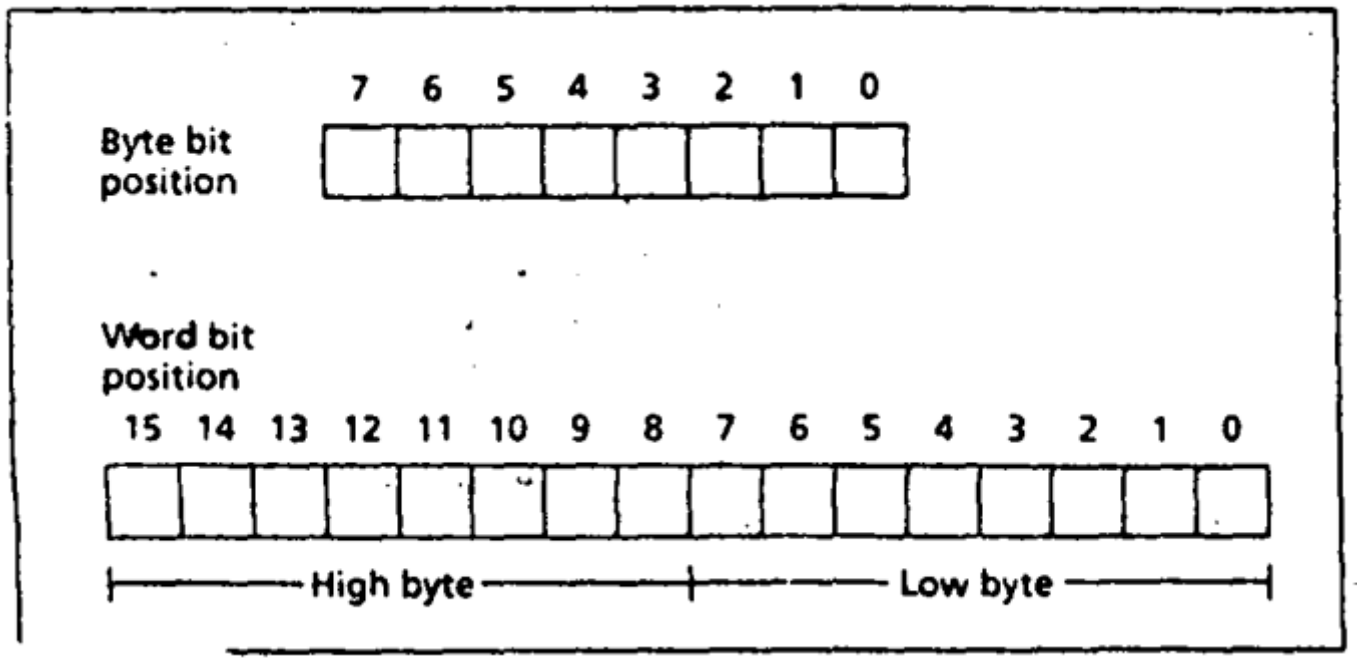

Fig 4: Bit Positions

RAM and ROM

RAM:

- Random Access Memory.
- Read and write operations.
- Program instructions and data are loaded here.
- Contents are lost when the machine is turned off.

ROM:

- Read Only Memory.
- Only read operation.
- Store system programs.
- Rom-based programs are called **firmware**.
- Firmware are responsible for loading **start-up programs** from ROM and **Self-testing** when turned on.

Buses

A processor communicates with memory and I/O circuits by using signals that travel along a set of wires or connections called buses.

There are three kinds of signals: **address, data, and control**.

And there are three buses:

- **Address bus**
- **Data bus**
- **Control bus**.

Buses (cont'd)

- To read the contents of a memory location, the CPU places the address of the memory location on the address bus, and it receives the data, sent by the memory circuits, on the data bus.
- A control signal is required to inform the memory to perform a read operation.
- The CPU sends the control signal on the control bus

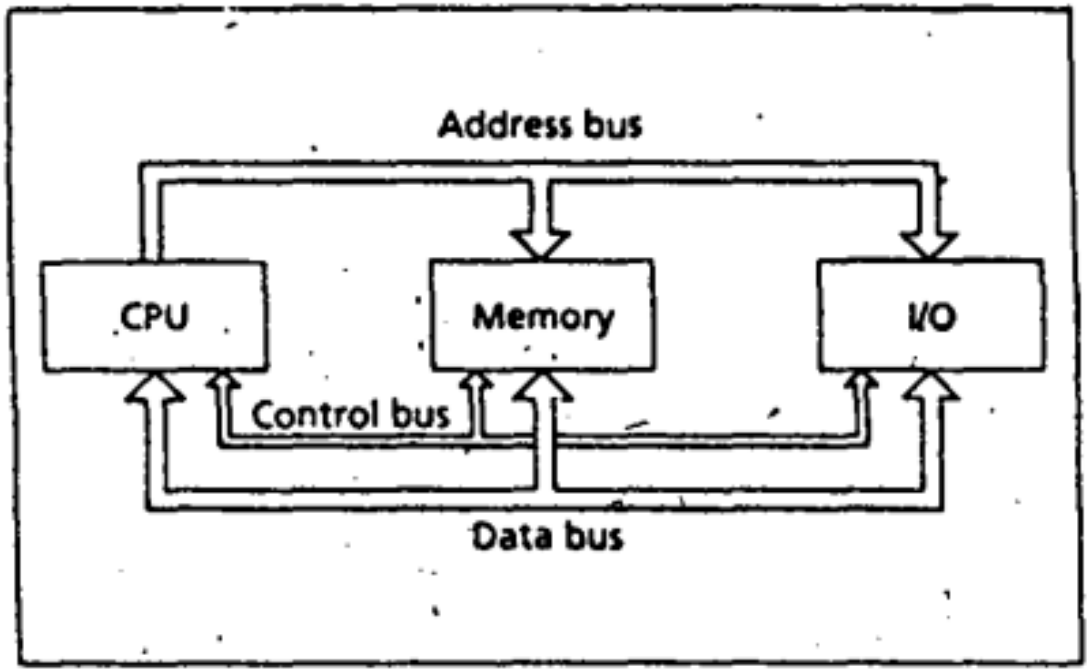

Fig 5: Bus Connections

Intel 8086 Microprocessor Organization

There are two main components:

- Execution unit
- Bus interface unit

The instruction performed by a CPU are called **Instruction Set**. Instruction Sets are:

- Simple
- Unique

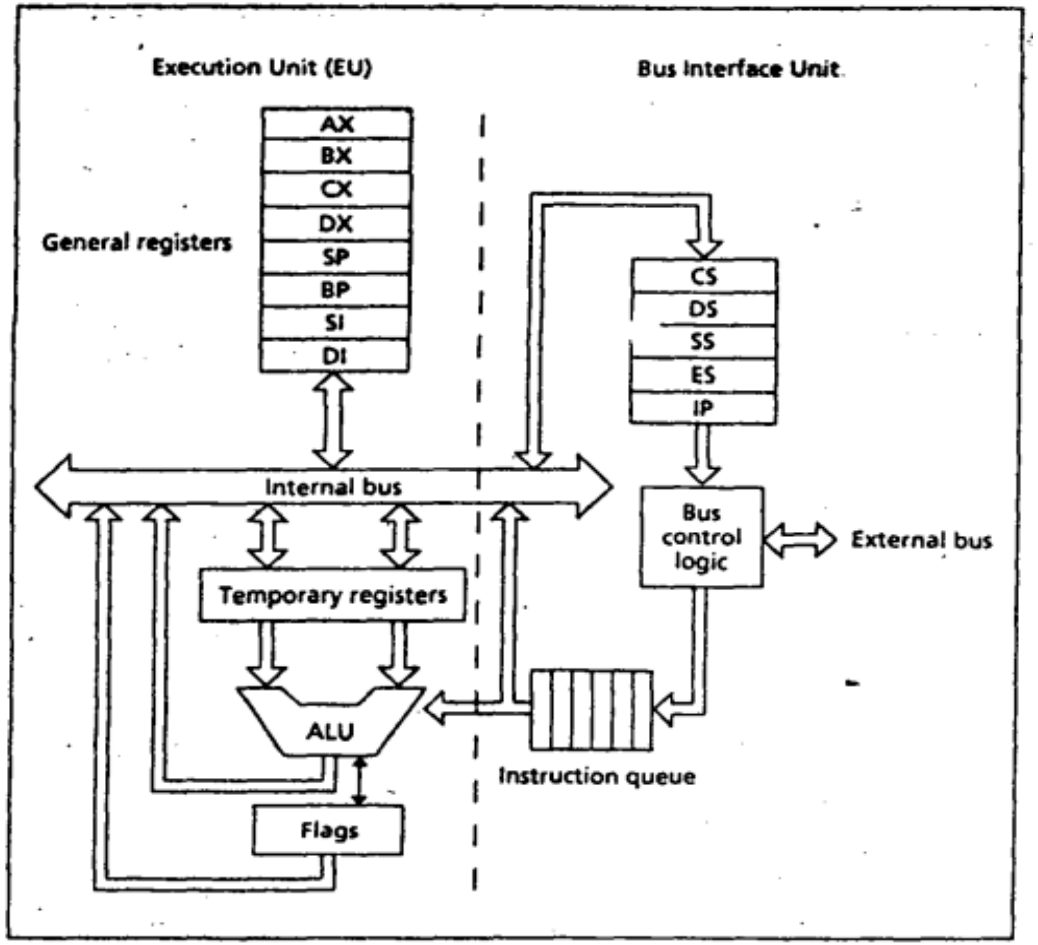

Fig 6: 8086 Organization

Execution Unit (EU)

- The purpose of the EU is to execute instructions.
- The **arithmetic and logic unit (ALU)** can perform arithmetic (+, -, x, /) and logic (AND, OR, NOT) operations.
- The data for the operations are stored in circuits called **registers**.
- The EU has 8 general registers: **AX, BX, CX, DX, SP, BP, SI, DI**.
- The EU contains **temporary registers** for holding operands for the ALU and **flag registers** whose individual bits reflect the results of a computation.

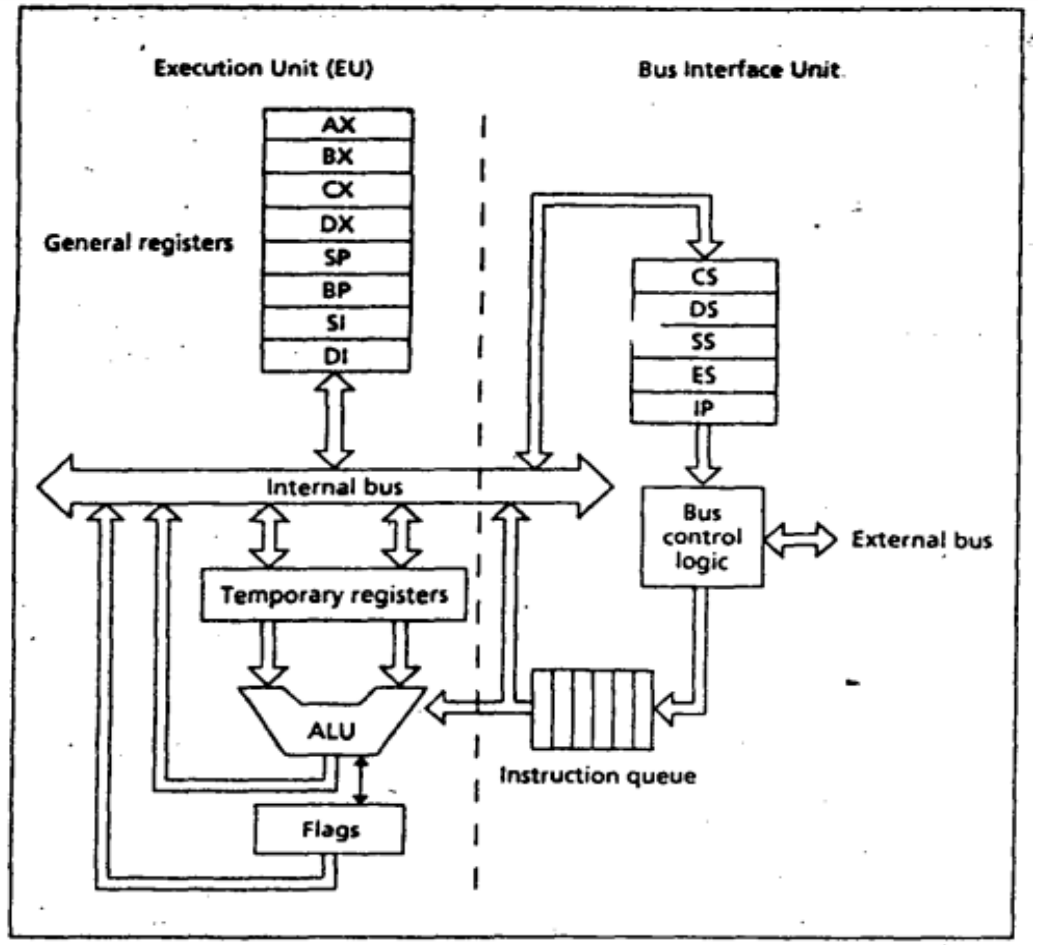

Fig 6: 8086 Organization

Bus Interface Unit (BIU)

- The BIU facilitates communication between the EU and the memory or I/O circuits.
- The BIU is responsible for transmitting addresses, data, and control signals on the buses.
- Registers: **CS, DS, SS, ES, IP**.
- The **instruction pointer (IP)** contains the address of the next instruction to be executed by the EU.
- EU and BIU are connected by an Internal Bus.
- While the EU is executing an instruction, the BIU fetches up to **six bytes** of the next instruction and places them in the **instruction queue**. This operation is called **instruction prefetch.**

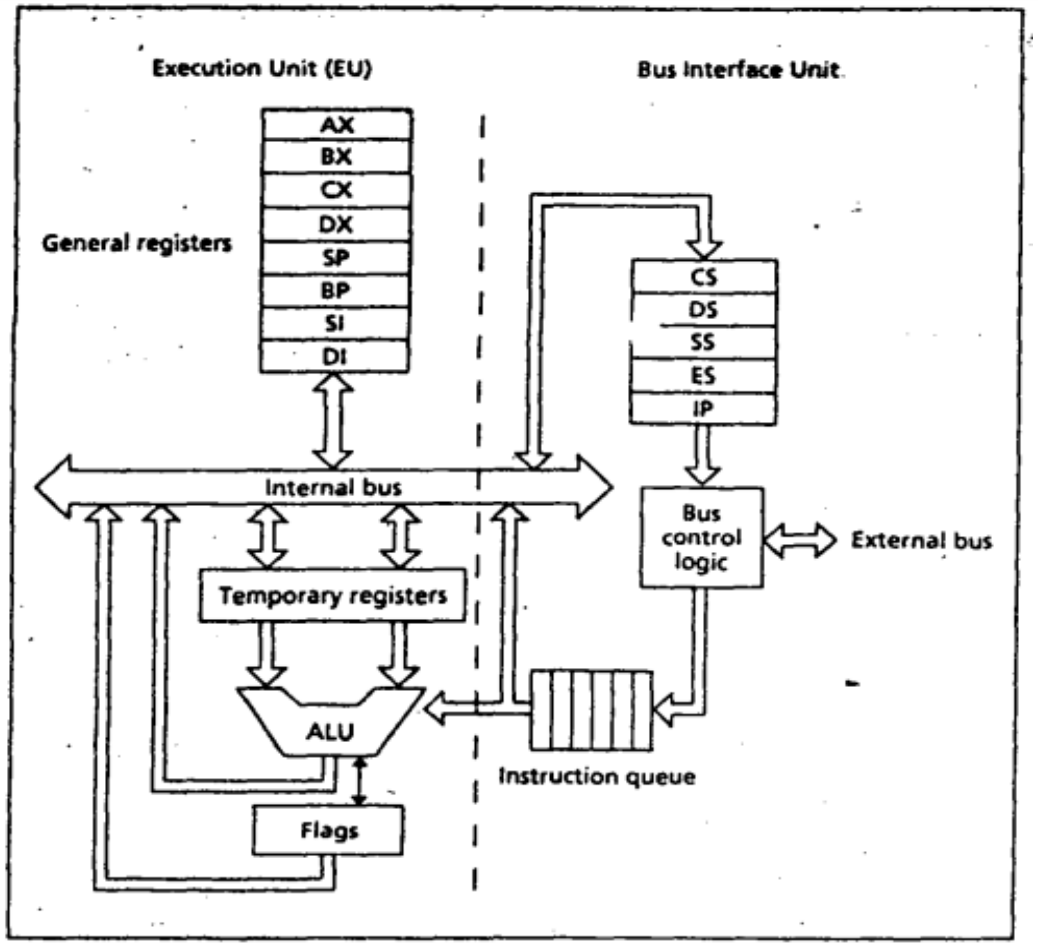

Fig 6: 8086 Organization

I/O Ports

Serial Ports:

- Data transfer 1 bit at a time.
- Slower.
- Requires less wiring connections.
- Slow devise like **keyboard** are connected.

Parallel Ports:

- Data transfer 8 / 16 bits at a time.
- Faster.
- Requires more wiring connections.
- Fast devices like **disk drivers** are connected.

*Devices like **printer**, can connected to either **serial or parallel** ports.*

Machine Instruction

The machine instruction has two parts:

- **Opcode:** Specifies the type of operation.
- **Operand:** Given as memory addresses to the data to be operated on.

The Instruction: ***MOV A, 42***

Opcode: **MOV** (This opcode specifies the operation to be performed: **Moving Data**)

Operand: **A** (This is a register where the data will be moved to / **Destination**)

Operand: **42** (This is the data that will be moved / **Source**)

Fetch-Execute Cycle

Fetch:

- Fetch an instruction from memory.
- Decode the instruction to determine the operation.
- Fetch data from memory if necessary.

Execute:

- Perform the operation on the data.
- Store the result in memory if needed.

Machine Language

- The operations of the computer's hardware are controlled by its software. When the computer is on, it is always in the process of executing instructions.
- The CPU can only execute **machine language instructions**.
- They are called **bit strings**.

Machine Instruction	Operation
10100001 00000000 00000000	Fetch the contents of memory word 0 and put it in register AX.
00000101 00000100 00000000	Add 4 to AX.
10100011 00000000 00000000	Store the contents of AX in memory word 0.

Assembly Language

- A more convenient language to use.
- We use symbolic names to represent operations, registers, and memory locations.
- If **location 0 is symbolized by A**, the preceding program expressed in IBM PC assembly language would look like this:

Assembly Instruction	Machine Instruction	Operation
MOV AX, A	10100001 00000000 00000000	Fetch the contents of location A and put it in register AX.
ADD AX, 4	00000101 00000100 00000000	Add 4 to AX.
MOV A, AX	10100011 00000000 00000000	Store the contents of AX into location A.

High-Level Languages vs Assembly Language

High Level

- C++, Python, Java etc.
- Closer to natural languages.
- Easier to read and understand.
- Requires less statements.
- Less time needed to code.
- Can be executed on any machine having a compiler for that language.

Assembly:

- Efficient because close to machine language.
- Faster.
- Produces shorter machine language program.
- Operations like reading or writing to specific memory locations and I/O ports can be done easily.